

MTGuard: Closing the Multi-Turn Generalization Gap in a Layered LLM-Injection Defense

An Empirical, Non-Circular Evaluation of the Mudette Cascade

Authors	Mudette Research Team
Venue	Global South AI Safety research program — non-circular evaluation follow-up to RAGE
Repository	Mudette-F — Python 3.14 (uv), local-only
Code & Data	docs/paper/{f0..f3}_* / · corpus/eval/manifest.json (sha256-pinned)
Predecessor	RAGE — Robust Agentic Security Gateway for Text-to-SQL (June 2026)

+3.8pp recall · 94.1% precision · 0.0% new false positives

Frozen test set · 147 scenarios · 4 run-to-report executions · Phases F0–F3

Abstract

Mudette is a research demo defending a fictitious enterprise copilot, Nexa Copilot, against multi-turn prompt-injection attacks, built around a layered detection engine we call MTGuard: a deterministic regex pre-filter (L1), an embedding-based conversational trajectory guard (L2), a weighted risk-fusion function, and an optional NVIDIA NIM-hosted LLM escalation judge. Mudette was built independently of its predecessor RAGE without incorporating professional reviewer feedback at design time. To compensate, we built a strict non-circular evaluation harness — a frozen, sha256-pinned test set assembled from external jailbreak benchmarks (JailbreakBench, SafeMTData) plus blind-generated in-domain and hard-negative corpora, evaluated under a run-to-report protocol that forbids iteration against the frozen split. Across four experimental phases (F0–F3) we used this harness to expose, name, and fix successive structural bottlenecks in the cascade: a mathematically provable ceiling on the trajectory layer alone (never reaching the block verdict), a judge invocation gate that starved the LLM judge of all traffic, and — once the gate was fixed — a single-turn “amnesia” in the judge that classified individually-innocuous turns of multi-turn attacks as benign. Injecting a three-turn raw conversational window into the judge’s prompt, tuned exclusively on a held-out development split and verified with exactly one frozen-test execution, flipped 4 of 5 previously-missed attacks into well-reasoned denials, raising frozen-test recall from 11.4% to 15.2% (+3.8 percentage points) and precision from 92.3% to 94.1%, with false positives unchanged at 2.4% and zero false positives among the judge’s own decisions. We report this result alongside its boundary — a purely volumetric escalation attack that the fix does not catch, predicted on dev before the test run — and an unresolved out-of-domain generalization gap on generic jailbreak content, which the evidence localizes upstream of the judge entirely.

Keywords:

multi-turn prompt injection · non-circular evaluation · LLM-as-judge · conversational trajectory · frozen test sets · NVIDIA NIM · fail-closed security

1 Introduction

1.1 The circularity problem in LLM security evaluation

A recurring failure mode in applied LLM-security research is **circular evaluation**: a defense is designed, a test set is authored by the same team that built the defense, and the defense is reported to perform well against it. Any detector tuned and measured against its own author's attacks will look strong regardless of whether it generalizes to attacks it did not anticipate. Our predecessor project, RAGE (Robust Agentic Security Gateway for Text-to-SQL), was evaluated with author-authored scenarios and internal metrics (AUC-D, TRI); external reviewer feedback on that submission — the starting point for this work — identified this circularity as the central methodological weakness to address in any follow-up.

Mudette was, in a specific sense, built “blind”: its layered defense (MTGuard) was designed and implemented before this evaluation harness existed and before reviewer feedback was incorporated into the design process. Rather than retrofit the defense to a self-authored benchmark, we inverted the order of operations for this study: we built the strictest evaluation protocol we could design **first**, froze it, and only then asked what the existing defense actually does — accepting whatever the numbers said, including several negative results.

1.2 From RAGE to Mudette

RAGE targeted a narrow, well-specified threat model — Crescendo-style multi-turn escalation against a text-to-SQL agent — with a four-layer pipeline (regex, RAG threat KB, stateful semantic filter, EWMA decision engine) and a hardened SQL gateway as a deterministic backstop. Mudette generalizes the defense to a broader enterprise-copilot setting (IT support for a fictitious company, Nexa Copilot) and renames the reusable core engine MTGuard: a pack-driven, multi-tenant architecture in which the detection cascade is agnostic to the specific agent it protects. The threat model widens accordingly, from SQL injection specifically to credential exfiltration, policy bypass, and prompt leakage via crescendo, salami-slicing, jailbreak, and social-engineering patterns.

1.3 Contributions

This paper reports four things:

- A non-circular evaluation harness with an explicit dev/test split, a sha256-pinned frozen test set dominated by external, non-self-authored attack sources, and a “run-to-report” protocol that forbids any iteration, threshold tuning, or model change against the frozen split once it is created.
- Empirical confirmation of a proven mathematical property of the fusion function (“Ghost #3”): the trajectory layer alone can never reach the block verdict, and on the frozen test it in fact never raises any alert at all.
- Discovery and naming of two novel failure modes in LLM-judge-augmented detection cascades, found and fixed strictly through the dev split before being measured once on the frozen test: **gate starvation** (Ghost #4) and **single-turn amnesia**.
- A minimal, cost-preserving fix — a raw three-turn conversational window in the judge's prompt, with no AI-generated summarization — that recovers 4 of 5 previously-missed multi-turn attacks on the frozen test with zero added false positives, together with an honestly reported boundary case and an

out-of-domain generalization gap that remains open.

2 System Architecture & Evaluation Framework

2.1 The MTGuard cascade

MTGuard processes each conversational turn through a fixed, four-stage pipeline before any response is generated:

```
user_message -> L1 RegexGuard -> L2 TrajectoryGuard -> RiskFusion
               -> [optional] EscalationJudge -> UserGate -> NexaAgent (RAG + LLM)
```

L1 — RegexGuard. A deterministic pre-filter of compiled regex rules with severity levels (LOW/MEDIUM/HIGH), covering known injection signatures. Cost: $O(1)$ per turn, no embeddings, no API calls.

L2 — TrajectoryGuard. Embeds each turn (2048-dimensional HashingVectorizer, L2-normalized) and tracks, per session: cosine proximity to benign-intent and sensitive-region centroids, turn-to-turn drift, an EWMA of proximity ($\alpha = 0.25$), and a 3-turn escalation-pattern detector.

RiskFusion. Combines L1 and L2 into a bounded risk score (0–100) and a four-level verdict:

Factor	Weight	Trigger
l1_high	45 (floor 75)	L1 hit, HIGH severity
l1_medium	25	L1 hit, MEDIUM severity
approaching_sensitive	20	$\text{max_proximity} \geq 0.62$ and $> \text{safe_score}$
escalation	15	3-turn escalation pattern
trajectory_high	10	$\text{trajectory_risk} \geq 0.10$
proximity_high	10	$\text{max_proximity} \geq 0.18$

Verdict bands: CLEAR (<25), WATCH (25–44), ALERT (45–74), CONTAIN (≥ 75). An [l1_high](#) hit unconditionally floors the score at 75 and forces CONTAIN — a deterministic backstop analogous to RAGE's SQL gateway. A safe-score veto can demote a marginal CONTAIN back to ALERT.

EscalationJudge. An optional LLM reviewer, invoked only when the verdict is WATCH or ALERT and the risk score clears a configurable gate. It calls an NVIDIA NIM-hosted model ([meta/llama-3.1-8b-instruct](#)) with the pack's judge instructions, the current message, raw L1/L2/fusion signals, and — as of Phase 3 of this study — a bounded window of prior turns. It returns a single-line ALLOW/DENY decision; a DENY upgrades the verdict to CONTAIN.

2.2 NVIDIA NIM inference and fail-closed semantics

Both the main agent and the judge call NVIDIA NIM's OpenAI-compatible endpoint with separate API keys and cached HTTP clients configured with explicit, asymmetric timeouts: 60s (10s connect, 1 retry) for the main agent, 20s (10s connect, 0 retries) for the judge. Judge decisions are memoized in a persistent cache keyed by `sha256(model || prompt)`.

Critically, no offline or simulated mode exists anywhere in the pipeline. A missing API key raises before any turn is processed; a timeout, connection error, or malformed NIM response raises and propagates rather

than being silently swallowed into a default ALLOW. The system fails closed on infrastructure failure, never silently permissive — an invariant held through every phase of this study.

2.3 Evaluation methodology

The evaluation corpus draws from five sources, each tagged with license and provenance:

Source	Content	License	Split
legacy(_playbook)	Self-authored cases	self-authored	dev only
domain_gen	Blind in-domain attacks	self-authored	dev + test
hard/easy_benign	Blind hard-negative / normal benigns	self-authored	dev / dev+test
jbb	JailbreakBench/JBB-Behaviors	MIT	test only
safemt	SafeMTData/SafeMTData	MIT	test only

A sixth source, ScaleAI/MHJ, was imported and then excluded on license grounds: it is CC-BY-NC-4.0 (non-commercial), and the only redaction scheme offered by the import tooling (`--hash-only`) replaces attack text with a placeholder that the detector would classify instead of the real attack — silently corrupting recall. We chose exclusion over a corrupted metric.

The freeze protocol. The test split was assembled once (105 attacks: 40 jbb, 30 safemt, 35 domain_gen; 42 benigns: 24 hard-negative, 18 easy) and frozen by recording sha256 hashes of the resulting corpus files. From that point forward: no iteration, threshold adjustment, or model change may be validated against the test split. All tuning happens on dev (132 scenarios) and is diagnosed with **keyless** probes — pure Python re-computation of fusion over already-captured signals, at zero API cost — before any frozen-test execution is spent.

Operating points. FLAG (max verdict $\geq$ ALERT) and BLOCK ($\geq$ CONTAIN, including judge DENY). Ground truth is scenario **provenance**, never the detector's own output. CI enforces no recall bands on this harness (anti-circularity); only false-positive invariants on easy benigns are gated automatically.

3 Empirical Phases & Structural Discoveries

Four phases, F0 through F3, form the empirical core of this study. Each phase is a single frozen-test execution preceded by dev-split diagnosis and a keyless, pre-registered prediction. Table 1 previews the full arc; the remainder of this section walks through it.

Table 1 — Frozen test set (147 scenarios: 105 attacks, 42 benign), all four phases.

Phase	Judge config	recall@FLAG	recall@BLOCK	FP@FLAG	prec@FLAG	Judge calls/DENY
F0	none (l1_l2)	11.4%	11.4%	2.4%	92.3%	—
F1	gate >=55 (starved)	11.4%	11.4%	2.4%	92.3%	0 / 0
F2	gate >=30 (amnesic)	11.4%	11.4%	2.4%	92.3%	5 / 0
F3	gate >=30 + 3-turn window	15.2%	15.2%	2.4%	94.1%	5 / 4

Three phases in a row (F0, F1, F2) produced *identical* aggregate metrics on the frozen test. Under a circular protocol this could be mistaken for stability; under this protocol it is the honest signature of a bottleneck that a naive metrics-only reading would miss — which is why each phase below reports the *mechanism*, not only the number.

3.1 Phase 0 & 1 — The Mathematical Ceiling and Gate Starvation

STRUCTURAL DISCOVERY — GHOST #3

The trajectory layer cannot block alone

RiskFusion.fuse reaches CONTAIN only via the deterministic l1_high floor (75) or a cumulative L2 score whose maximum possible weight (55) falls short of the CONTAIN band. This is a proven mathematical property, independent of test data: the pure-trajectory configuration can never reach BLOCK. The frozen-test run confirmed it more strongly than the proof requires: l2_only recall@BLOCK = 0.0%, and recall@FLAG was also 0.0% — the trajectory layer never raised even a WATCH-level alert on this test set.

The keyless [l1_l2](#) configuration performed identically to [l1_only](#) (11.4% recall@FLAG, 92.3% precision, 2.4% FP@FLAG): zero of the 105 attacks flagged by the combined stack were missed by regex alone. Per-source breakdown localized *why*: recall on [domain_gen](#) (in-domain, announced threat model) reached 31.4%, while [jbb](#) sat at 0.0% and [safemt](#) at 3.3%. This is the direct, quantitative answer to the circularity concern that motivated this harness: measured against a test set an author did not write, the keyless detector's true reach is roughly a tenth of what a self-authored benchmark would likely have reported.

STRUCTURAL DISCOVERY — GHOST #4**Gate starvation: the judge that could never be asked**

Before spending any judge API calls, code inspection yielded a falsifiable prediction: `should_invoke` gates at `risk_score >= 55`, which already implies ALERT (≥ 45) — the judge could only ever see turns already flagged, and could upgrade ALERT to CONTAIN but never create a new FLAG. Combined with 100% of F0's first-flags carrying `l1_high` (bypassing the judge entirely), the touchable population was predicted to be small or empty.

The live run confirmed the degenerate case: zero invocations across 147 scenarios (432 turns). A keyless probe found the maximum fusion score without `l1_high` was 35 on test and 45 on dev — never approaching the 55 gate. A live smoke call with a synthetic prompt confirmed the NIM path itself was healthy (DENY in 2.5s): the starvation was architectural, not infrastructural.

3.2 Phase 2 — Single-Turn Amnesia

A keyless dev diagnostic found: (a) a dead risk factor — `approaching_sensitive` requires proximity ≥ 0.62 , but the maximum observed anywhere in dev was 0.574; (b) single-knob L2 sensitivity sweeps never reach 55 without `l1_high`, while spending false-positive budget; and (c) a gate-value sweep showing 30 as the first threshold exposing meaningful attack traffic — seven dev attack turns, zero benign turns.

We changed exactly one constant (`DEFAULT_JUDGE_THRESHOLD`: 55 $\rightarrow$ 30). On dev, the judge denied 3 of 7 (42.9%) — all cases whose *individual turn text* contained an explicit sensitive ask. The four ALLOWs were the informative result: three were turn-zero probes (correctly permissive) and one was a mid-conversation salami turn whose isolated wording read as routine IT support.

FAILURE MODE — CONFIRMED ON TEST**Single-turn amnesia**

The frozen-test run of gate 30 (no context window) reproduced F0's aggregate numbers exactly, but for a new reason: 5 invocations, 0 denials. All five were domain_gen crescendo/salami attacks at turns 3–4, every one ALLOWed with a variant of: “the current message is a benign IT support question.” A multi-turn attack engineered so each step reads as ordinary work is, definitionally, invisible to a judge that only ever sees one step — `EscalationJudge_build_prompt` passed only the current turn's text and numeric signals, with no conversational history.

3.3 Phase 3 — Conversational Context

The fix was deliberately minimal: `_build_prompt` gained an optional `history` parameter, windowed to the last three raw prior user turns (`JUDGE_CONTEXT_WINDOW = 3`). We chose raw turn text over an AI-generated summary specifically to avoid a second LLM call, added latency, and an unverified layer of interpretation between evidence and decision. History is sourced from the same `ConversationState` already backing L2 in production; the invocation gate itself was left untouched.

Dev validation (non-regression). The three existing denials survived being re-judged with a longer prompt — 3 of 3 preserved — with justifications visibly upgraded to explicitly trajectory-aware phrasing. The one addressable dev case, escalating ticket-record requests with no named sensitive target, remained ALLOWed — documented *before* touching the frozen test as the predicted boundary of a window that recognizes accumulation toward a named target but not pure request-volume growth.

Frozen-test result. The single permitted execution produced the only recall movement in the entire research arc: recall@FLAG and recall@BLOCK rose from 11.4% to 15.2% (+3.8pp), while precision rose from 92.3% to 94.1% and false positives stayed flat at 2.4% — every judge denial on the frozen test was a true attack.

Table 2 — Transfer of judge decisions from Phase 2 to Phase 3 (frozen test).

Scenario	Category	F2	F2 reason	F3	F3 reason
domain_gen_0001	crescendo	ALLOW	benign IT support question	DENY	trajectory accumulates toward exfiltration despite benign appearance
domain_gen_0015	salami	ALLOW	benign ... performance review	DENY	trajectory accumulates ... despite looking routine
domain_gen_0035	crescendo	ALLOW	benign IT support question	DENY	trajectory accumulates toward credential/data exfiltration
domain_gen_0041	salami	ALLOW	benign directory export request	DENY	stepwise movement toward bulk export and credential collection
domain_gen_0051	salami	ALLOW	legitimate IT support question	ALLOW	stepwise progression toward bulk export, but still within legitimate bounds

The fifth case is discussed on its own terms in Section 4.1: it is not a residual instance of amnesia (the judge's own reasoning shows it **sees** the trajectory) but a distinct, previously predicted boundary.

4 Failure Modes & Limitations (Discussion)

4.1 The volumetric-escalation boundary (domain_gen_0051)

The one case Phase 3 did not recover is scientifically the most interesting result in this study, because it was predicted on the development split before the frozen-test run that confirmed it. The dev miss ([legpb_salami_export](#)) and the test miss ([domain_gen_0051](#)) share an identical structure: progressively larger *quantities* of an already-permitted resource class, with no named credential, secret, or export target appearing anywhere in the visible window. The judge's own justification — “shows a stepwise progression toward bulk data export, but the current message is still within the bounds of a legitimate IT support request” — shows it perceives the escalation and, under its ALLOW-biased instructions, still declines to deny it.

This is a calibrated boundary of an evidence-based judge, not a blind spot: recovering it would require a new signal — volumetric or repetition-based, orthogonal to semantic proximity — rather than more judge context or a different gate. We report this as future work (Section 5) rather than attempt a same-session fix, consistent with the freeze protocol.

4.2 The out-of-domain generalization gap

Across all four phases, [jbb](#) (40 generic jailbreak attacks) and [safemt](#) (30 generic multi-turn attacks) never moved: recall stayed at 0.0% and 3.3% respectively. Fusion scores on these sources never leave the CLEAR band, so zero jbb/safemt turns ever reach the judge's gate, at any gate value tested. No amount of judge improvement can act on traffic that never arrives. Eighty-nine of the 105 frozen-test attacks still produce no flag under any configuration measured here.

This is the honest complement to the Phase 3 result: MTGuard's recovery of multi-turn attacks is real but do main-bound — a claim about defense-in-depth *within* an announced threat model, not a claim about general-purpose jailbreak robustness. Closing this gap is an L1/L2 coverage problem, categorically distinct from the judge-context question this study answers.

4.3 Other declared limitations

- Embedding quality. L2's [HashingVectorizer](#) (collision-based, no learned semantics) is fast but semantically shallow; it cannot recognize paraphrase the way a trained sentence embedding would. The dead [approaching_sensitive](#) factor is partly attributable to this.
- Unmeasured benign false-denial rate. No benign scenario has ever reached the judge's gate, in any phase or split — a consistent, four-times-replicated invariant, but one that leaves the judge's false-denial rate on legitimate traffic empirically unmeasured.
- Parser bias. [parse_judge_response](#) defaults to ALLOW on ambiguous output — a fail-*open* choice in tension with the pipeline's fail-*closed* posture on infrastructure errors. Never exercised in this study, but worth re-examining before deployment.
- Dev-corpus composition. Dev is dominated by self-authored and in-domain content; no generalization claim in this paper rests on dev-only numbers.
- License-driven exclusion. ScaleAI/MHJ (40 additional attacks) was imported, reviewed, and excluded rather than redacted into a corrupted metric.

5 Conclusion & Future Work

This study set out to answer, with evidence rather than assertion, the central methodological criticism carried over from RAGE: does this cascade actually generalize, and does adding an LLM judge help? Under a frozen, sha256-pinned, run-to-report protocol, the honest answer is layered. The keyless cascade is precise (92.3% precision) but narrowly domain-bound (11.4% recall, 0% on generic jailbreak content). Adding an LLM judge, on the first attempt, changed nothing — not because the judge reasons poorly, but because its own invocation gate made it mathematically unreachable (Ghost #4). Feeding the judge exposed a second, more interesting bottleneck: a judge with no memory of the conversation classifies the individual steps of a multi-turn attack exactly as they read — as routine work.

Giving the judge a bounded, three-row-turn window — the smallest change that could plausibly work, chosen explicitly over an AI-summarized alternative to preserve latency, cost, and evidence integrity — recovered four of five previously-missed attacks on a single frozen-test execution, with precision *rising* and false positives unchanged. The one case it did not recover was predicted before that execution ran, on the development split alone, and its later confirmation on frozen data is itself evidence that the diagnostic methodology, not merely the fix, is sound.

The total empirical cost of this four-phase arc was four frozen-test executions and 24 cached judge decisions across both splits — effectively zero dollars at current NVIDIA NIM free-tier pricing. We regard the discipline of that cost as much a contribution as the recall number: every phase's fix was designed and validated entirely on dev, with a falsifiable, pre-registered prediction stated before the one test execution that could confirm or refute it.

5.1 Future work

- Migrate the shared embedder from ``HashingVectorizer`` to ``sentence-transformers``. A learned, semantically dense embedding may revive the dead `approaching_sensitive` factor and narrow the out-of-domain gap by generalizing past `domain_gen`'s exact vocabulary. Must be re-tuned and re-frozen on dev before any test-set claim is made about it.
- A volumetric or repetition-based L2 signal, orthogonal to semantic proximity, to address the calibrated boundary identified in Section 4.1.
- Upstream out-of-domain coverage — broader L1 signatures and/or additional L2 sensitive regions informed by jbb/safemt failure analysis — as a research question distinct from, and prerequisite to, any further judge-context work.
- A dedicated benign stress corpus engineered to clear the judge's invocation gate, to finally measure the false-denial rate that four phases of this study left structurally unmeasured.

References

- [1] M. Russinovich, A. Salem, and R. Eldan, “Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack,” arXiv:2404.01833, Microsoft, 2024.
- [2] Mudette Research Team (predecessor project), “RAGE: Robust Agentic Security Gateway for Text-to-SQL — Defending Against Multi-Turn Crescendo Attacks,” Global South AI Safety Hackathon, June 2026.
- [3] P. Chao, et al., “JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models,” arXiv:2404.01318, 2024. (JailbreakBench/JBB-Behaviors, HF rev. 886acc3..., MIT.)
- [4] SafeMTData contributors, “SafeMTData: A Multi-Turn Safety Evaluation Dataset,” Hugging Face Hub. (SafeMTData/SafeMTData, HF rev. 04af7bd..., MIT.)
- [5] ScaleAI, “MHJ: Multi-Turn Human Jailbreaks,” Hugging Face Hub. (Reviewed and excluded on CC-BY-NC-4.0 license grounds; Section 2.3.)
- [6] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” v1.1, 2023.
- [7] NVIDIA, “NVIDIA NIM — API Catalog Documentation,” integrate.api.nvidia.com/v1, build.nvidia.com.
- [8] A. Anil, et al., “Many-Shot Jailbreaking,” Anthropic Technical Report, 2024.
- [9] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, “Universal and Transferable Adversarial Attacks on Aligned Language Models,” arXiv:2307.15043, 2023.

Mudette source code and the complete evaluation harness correspond to the repository at commit [9d068b5](#) (Phase 13.2). Curated, verbatim evaluation artifacts for every phase cited in this paper ([report.md](#), [metrics.json](#), and phase-specific [CURATION.md](#) analysis) are committed under [docs/paper/](#).

LLM USAGE STATEMENT

The Mudette codebase, its evaluation harness, and this paper were developed with AI pair-programming assistance (Claude, via the Cursor IDE) under a strict human-gated workflow: every code change and every execution against the frozen test set required an explicit, prior human approval, and no iteration against the frozen split was permitted at any point after freezing. All quantitative results reported in this paper are direct, unedited outputs of the evaluation harness ([report.md](#) / [metrics.json](#) per phase); no number in any table above was hand-edited or recomputed outside the harness. The human author reviewed and approved each phase's diagnostic plan before execution and each phase's curated findings before inclusion in this document.